机器学习系统学习目录

结合你目前的基础——学过高数、线性代数，正在学习概率论，希望以后往 AI 方向发展——我建议按照下面的路线学习。

我们的目标不是记住大量公式，而是建立一条完整的知识主线：

数据 → 模型 → 损失函数 → 优化 → 评估 → 改进 → 深度学习。

整个学习分为 8 个阶段。

第一阶段：建立机器学习的整体认知

目标：理解机器学习在做什么，掌握最基本的概念。

课程	学习内容	掌握标准
1. 机器学习概述	机器学习是什么、基本流程	能讲清机器学习如何从数据中学习
2. 机器学习的类型	监督学习、无监督学习、强化学习	能区分不同任务
3. 数据集划分	训练集、验证集、测试集	理解各自的作用
4. 模型与参数	特征、标签、模型、参数	能区分输入、输出和模型内部参数
5. 欠拟合与过拟合	泛化能力、过拟合、欠拟合	能分析模型为什么表现不好

第二阶段：模型如何学习——机器学习的核心机制

目标：真正理解模型是如何通过训练找到合适参数的。

课程	学习内容	掌握标准
6. 线性回归	模型、权重、偏置	理解最简单的预测模型
7. 损失函数	MSE、MAE 等	理解如何衡量预测误差
8. 梯度与偏导数	参数变化如何影响损失	理解梯度的含义
9. 梯度下降	学习率、参数更新	理解模型如何逐步优化
10. 多样本训练	多个样本如何共同影响参数	理解真实训练过程
11. 批量梯度下降	Batch、Mini-batch、SGD	理解不同的参数更新方式
12. 优化算法	Momentum、RMSProp、Adam	理解常用优化方法的作用

这里有一条特别重要的主线：

$$ \boxed{ \text{模型预测} \rightarrow \text{计算损失} \rightarrow \text{计算梯度} \rightarrow \text{更新参数} \rightarrow \text{重复训练} } $$

当前进度：已学习线性回归、损失函数、梯度和梯度下降的基本思想，正在向分类模型过渡。

第三阶段：机器学习的经典模型

目标：掌握最常见的传统机器学习算法，理解它们分别适合解决什么问题。

课程	学习内容	掌握标准
13. 逻辑回归	Sigmoid、二分类、决策阈值	理解分类模型的基本原理
14. 分类损失函数	交叉熵、概率预测	理解分类模型如何衡量错误
15. 多分类问题	Softmax、多分类交叉熵	理解多个类别如何进行预测
16. K 近邻算法	距离、邻居投票	理解基于相似性的预测
17. 决策树	特征划分、信息增益、基尼指数	理解树模型如何划分数据
18. 随机森林	Bagging、特征随机选择	理解多个模型如何协作
19. 支持向量机	间隔、分类边界、核方法	理解经典分类算法的核心思想
20. K-means 聚类	簇、质心、迭代更新	理解无监督学习的典型方法
21. PCA 降维	特征压缩、方差、主成分	理解降维的基本目标

这部分不需要把每种算法的每个推导都背下来。

我们优先掌握三个问题：

1. 算法的核心思想是什么？
2. 它通过什么机制学习？
3. 它适合什么场景，有什么局限？

第四阶段：如何判断模型好不好，以及如何改进

目标：理解模型评估、过拟合和泛化问题。

课程	学习内容	掌握标准
22. 回归评估指标	MAE、MSE、RMSE、(R^2)	会解释常见指标
23. 分类评估指标	Accuracy、Precision、Recall、F1	能根据场景选择指标

课程	学习内容	掌握标准
24. 混淆矩阵	TP、FP、TN、FN	理解分类错误的类型
25. 数据泄漏	训练和测试信息污染	知道如何避免评估失真
26. 过拟合的解决办法	正则化、数据增强、早停	理解如何改善泛化
27. 偏差与方差	Bias-Variance Tradeoff	理解模型复杂度与泛化的关系
28. 特征工程	标准化、归一化、特征选择	理解数据处理为什么重要
29. 超参数调优	网格搜索、随机搜索、验证集	理解如何选择模型配置

这里要建立一个重要认识：

模型在训练集上表现好，不等于它学到了真正能够泛化的规律。

数据质量、模型复杂度、训练方法和评估方式都会影响最终效果。

第五阶段：数学基础与概率视角

这一阶段与前面内容穿插学习，不需要等到全部学完才开始。

目标：知道机器学习背后的数学在解决什么问题。

课程	学习内容	掌握标准
30. 线性代数回顾	向量、矩阵、矩阵乘法	理解多特征模型的计算
31. 导数与偏导数	导数、链式法则、梯度	理解参数更新的数学依据
32. 概率基础	随机变量、条件概率、期望、方差	理解概率预测
33. 常见概率分布	伯努利、二项、正态分布	理解常见数据与模型假设
34. 最大似然估计	似然、参数估计	理解模型如何利用数据估计参数
35. 贝叶斯思想	先验、似然、后验	建立概率推断的基本认识
36. 信息论基础	熵、交叉熵、KL 散度	理解分类损失和信息度量

考虑到你正在学概率论，我们会把这些知识和具体模型联系起来，而不是孤立地推公式。

例如，学习交叉熵时，再解释它与概率预测、最大似然估计之间的联系。

第六阶段：进入深度学习

这是从传统机器学习迈向现代 AI 的关键阶段。

目标：理解神经网络的基本结构、训练方法和常见架构。

课程	学习内容	掌握标准
37. 神经网络概述	神经元、层、权重、偏置	理解神经网络的基本组成
38. 激活函数	ReLU、Sigmoid、Tanh	理解为什么需要非线性
39. 前向传播	输入如何变成输出	能追踪网络的计算过程
40. 反向传播	链式法则、梯度传递	理解神经网络如何计算梯度
41. 神经网络训练	损失函数、优化器、训练循环	理解完整训练过程
42. 正则化与归一化	Dropout、权重衰减、BatchNorm、LayerNorm	理解训练稳定性和泛化
43. PyTorch 入门	Tensor、自动求导、模型训练	能阅读和编写基础训练代码
44. GPU 与训练效率	GPU 计算、显存、批量计算	理解深度学习的计算特点

在这一阶段，你会发现：

前面学的线性回归、梯度下降和损失函数并没有失去作用，它们仍然是理解神经网络的基础。

神经网络只是引入了更复杂的模型结构，训练的核心思想依然是计算损失并调整参数。

第七阶段：现代深度学习与 AI 方向

考虑到你未来想做 AI，这部分是重点。

课程	学习内容	掌握标准
45. CNN 卷积神经网络	卷积、特征图、池化	理解图像模型的基本思想
46. RNN 与序列建模	序列、隐藏状态	理解早期序列模型
47. Attention 机制	Query、Key、Value	理解注意力如何建立信息联系
48. Transformer	自注意力、前馈网络、残差连接	理解现代语言模型的基本架构
49. 大语言模型基础	Token、Embedding、预训练、微调	理解 LLM 的基本训练流程
50. 模型对齐与推理	指令微调、偏好优化、推理能力	了解模型如何适应任务
51. 生成模型	VAE、GAN、扩散模型	理解常见生成式 AI 方法

课程	学习内容	掌握标准
52. 强化学习基础	状态、动作、奖励、策略	理解智能体如何学习决策

对于你目前的阶段，不必马上深入所有方向。

建议优先掌握：

机器学习基础 → 深度学习 → PyTorch → Transformer → 大语言模型基础。

之后再根据兴趣选择计算机视觉、自然语言处理、强化学习或其他方向。

第八阶段：科研阅读、复现与实践

目标：从“理解知识”走向“能够参与科研”。

课程	学习内容	掌握标准
53. 数据集与实验设计	数据划分、基线、控制变量	能设计基本实验
54. 阅读论文	问题、方法、实验、结论	能抓住论文主线
55. 复现模型	环境、代码、训练、评估	能运行并验证已有方法
56. 实验分析	消融实验、指标对比、误差分析	能解释实验结果
57. 开源项目	Git、代码阅读、Issue、PR	能参与协作开发
58. 小型研究项目	研究问题、方法改进、实验验证	能完成初步研究工作

这一阶段不要求你立刻发表论文。

首先要能做到：读懂一篇入门论文，跑通官方代码，复现实验结果，并解释实验中观察到的现象

正文:

一、机器学习究竟在做什么?

机器学习的核心思想是：

让计算机从数据中学习规律，再用学到的规律处理没见过的数据。

机器学习的基本流程是：

数据 → 模型 → 预测 → 计算损失 → 调整参数 → 不断训练 → 评估效果。

二、监督学习、无监督学习和强化学习

根据学习方式的不同，机器学习可以分成三大类。

类型	核心思想	例子
监督学习	根据带有正确答案的数据学习	预测房价、识别垃圾邮件
无监督学习	从没有明确目标标签的数据中寻找规律	用户聚类、数据降维
强化学习	通过与环境交互，根据奖励学习行动策略	游戏智能体、机器人控制

其中，我们目前主要学习监督学习。

监督学习又有两个重要任务：

- **回归**：预测连续数值，例如房价、成绩。
- **分类**：预测类别，例如垃圾邮件识别、图像分类。

回归预测数值，分类预测类别。这是最基本的区分。

三、训练集、验证集和测试集

我们通常将数据划分为三部分：

数据集	作用	通俗理解
训练集	学习模型参数	做练习题
验证集	选择模型、调整超参数	用模拟考试检查学习效果
测试集	评估最终模型	最后的独立考试

这里引出一个重要概念：**泛化能力**。

泛化能力，就是模型处理没见过的新数据时，表现得怎么样。

如果模型在训练集上表现很好，在测试集上却表现很差，通常说明它出现了过拟合。

- 欠拟合：模型没有学好，连训练数据都处理不好。
- 过拟合：模型过度适应训练数据，对新数据表现不好。

所以，不能只看训练集的成绩来判断模型是否优秀。

四、模型和参数是什么？

我们之前学习了最简单的线性回归模型：

$$\hat{y} = wx + b$$

这里有三个重要概念：

- x：输入特征。
- $\hat{y}$：模型预测的结果。
- w, b：模型参数。

其中：

- w 决定输入变化时，预测结果如何变化。
- b 决定直线的整体位置。

例如：

$$\hat{y} = 2x + 10$$

当 x 增加 1 时，预测值增加 2。

如果把模型改成：

$$\hat{y} = 5x + 10$$

当 x 增加 1 时，预测值增加 5。

这说明参数不同，模型表现出来的规律也不同。

训练模型的一个核心任务，就是找到合适的参数。

当然，参数合适不代表模型结构一定合适。如果真实规律非常复杂，而我们只使用简单的直线，那么无论怎么调整参数，都可能无法充分拟合数据。

这就是为什么模型结构和模型参数需要区分。

五、损失函数：判断模型预测得怎么样

模型预测之后，我们需要知道预测得好不好。

这就需要损失函数。

例如，真实成绩是 90 分，模型预测 60 分，那么预测误差就比较大。

对于回归任务，我们可以使用平方误差：

$$L = (\hat{y} - y)^2$$

其中：

- $\hat{y}$ 是预测值。
- y 是真实值。
- L 是当前样本的损失。

预测越接近真实值，损失通常越小；预测偏差越大，损失通常越大。

多个样本的损失可以进一步求平均，得到均方误差 MSE。

损失函数负责衡量模型预测得有多差，但它本身并不负责调整模型参数。

不同任务通常会采用不同的损失函数。例如：

- 回归任务经常使用 MSE。
- 分类任务经常使用交叉熵。

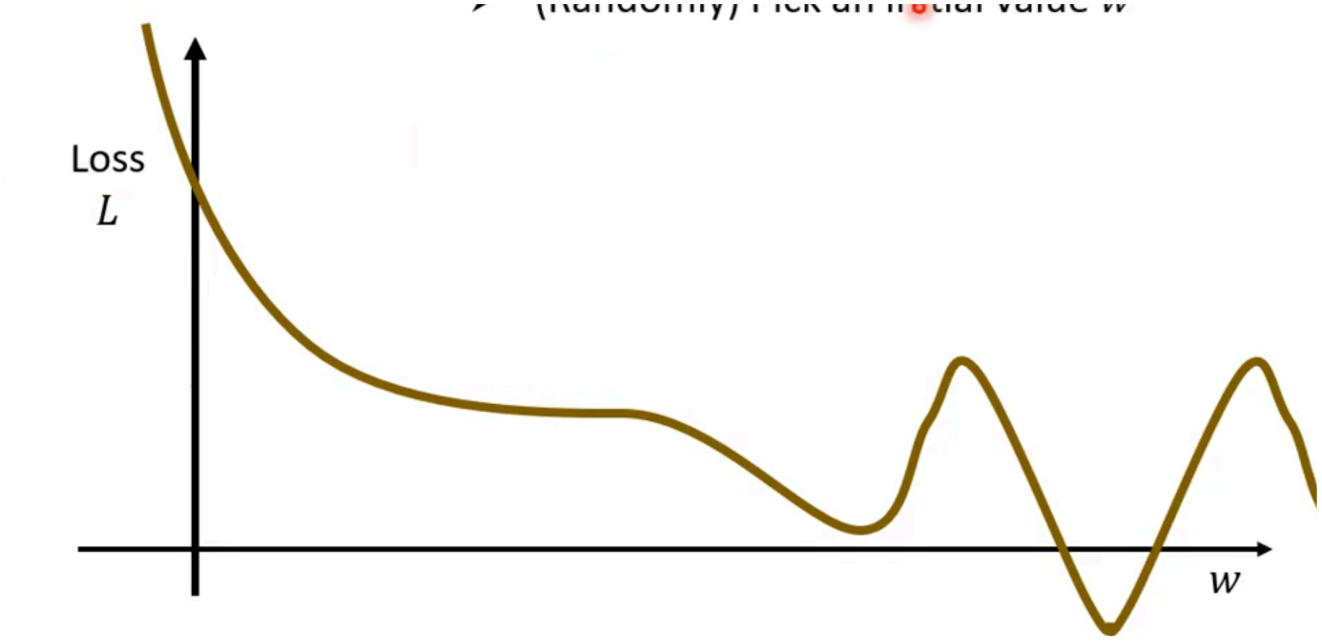
六、梯度下降：模型究竟怎样调整参数?

这是前面学习中最重要的一条主线。

既然损失函数告诉我们误差有多大，那么计算机究竟如何修改 (w) 和 (b)?

答案是：通过计算梯度，再使用梯度下降更新参数。

梯度 = 损失函数对每一个参数的偏导数，把它们拼成一个向量。 `(∂L/∂w, ∂L/∂b)`。



我们把三个概念分清楚。

概念	作用	通俗理解
模型	根据输入产生预测	做题的人
损失函数	衡量预测结果有多差	判卷打分
梯度	描述损失对参数变化的局部敏感程度和方向	提供调整方向的信息
梯度下降	根据梯度更新参数	按一定步幅调整做题方法

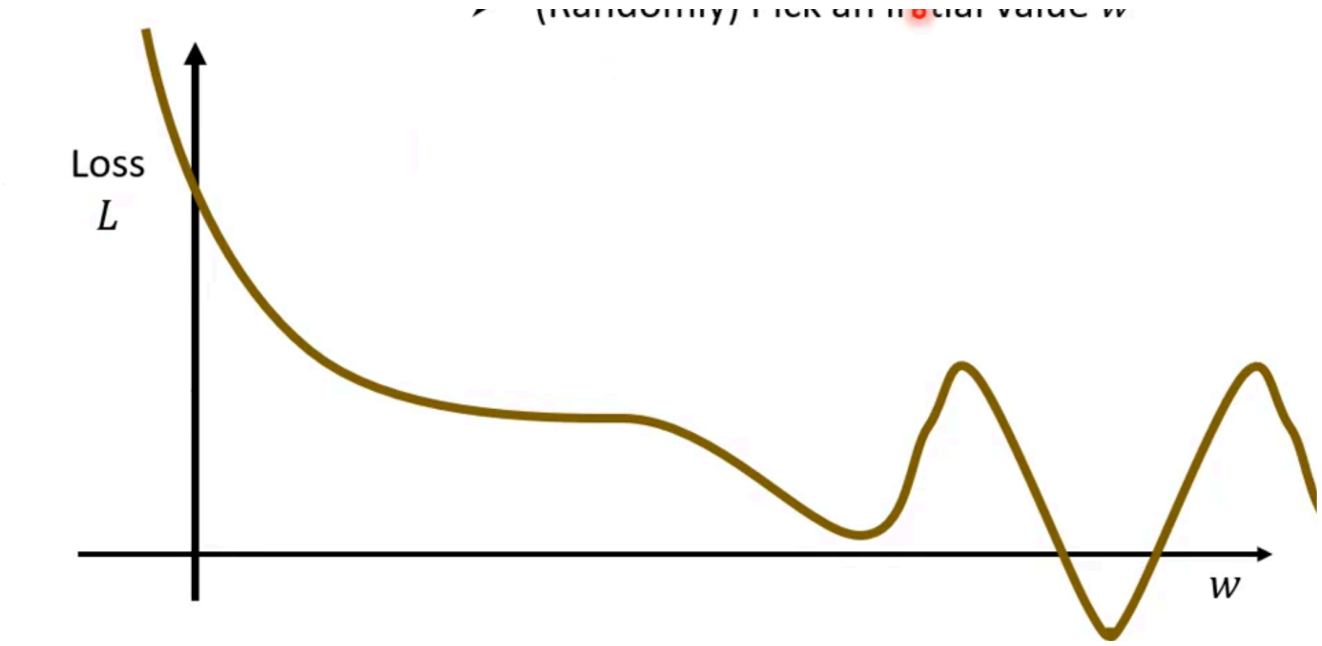
梯度下降的核心公式是：

$$\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta L}$$

这里的 θ 代表需要调整的参数，例如 w 或 b。

- θ_{old}：旧参数。
- θ_{new}：更新后的参数。
- η：学习率，决定更新步幅。
- $\nabla_\theta L$：损失对参数的梯度。

如果梯度为负，减去负数就会增大参数；如果梯度为正，减去正数就会减小参数。



不过，要注意：梯度反映的是当前参数位置附近的局部变化规律。学习率设置不当时，更新不一定能让损失降低。

因此，实际训练会反复执行：

预测 → 计算损失 → 计算梯度 → 更新参数。

不断重复这个过程，就是模型训练的核心机制之一。

七、逻辑回归：从预测数值到预测类别

前面学的线性回归适合预测连续数值。

但如果我们想识别垃圾邮件，就需要分类模型。

假设标签为：

$$y = \begin{cases} 1 & \text{垃圾邮件} \\ 0 & \text{正常邮件} \end{cases}$$

如果直接使用线性模型，输出可能是 1.7 或 -0.3，这些数值不能直接解释为合理的概率。

逻辑回归的做法是：

先进行线性计算：

$$z = wx + b$$

再通过 Sigmoid 函数，将结果转换为 0～1 之间的数：

$$p = \frac{1}{1 + e^{-z}}$$

这里的 p 是模型预测样本属于正类的概率估计。

例如：

- $p = 0.9$：模型认为样本很可能属于正类。
- $p = 0.2$：模型认为样本不太可能属于正类。

接着，我们可以设置分类阈值。

例如阈值为 0.5：

- $p \geq 0.5$，预测为正类。
- $p < 0.5$，预测为负类。

这里有一个容易混淆的地方：

Sigmoid 负责将线性输出转换为概率；分类阈值负责根据概率决定最终类别。

两者不是一回事。

另外，逻辑回归虽然名字中有“回归”，但它常常用于分类任务。

八、交叉熵：分类任务如何衡量预测好坏？

我们已经知道，逻辑回归会输出正类概率。

假设一封邮件的真实标签为 1，也就是垃圾邮件。

现在有两个模型：

- 模型 A：预测正类概率为 0.95。
- 模型 B：预测正类概率为 0.2。

显然，模型 A 更符合真实标签。

交叉熵就是一种常用于分类任务的损失函数。它会根据模型给真实类别分配的概率，计算相应的损失。

对于二分类任务：

$$\boxed{L = -[y \log p + (1-y) \log(1-p)]}$$

暂时不必死记公式，理解两种情况就够了。

如果真实标签是 1：

模型应该让 (p) 尽可能接近 1。

预测概率越低，损失越大；尤其当模型非常确信自己预测错了时，损失会很大。

如果真实标签是 0：

模型应该让 (p) 尽可能接近 0。

此时，预测概率越高，损失越大。

所以，交叉熵的核心思想是：

让模型给真实类别分配较高的概率，并惩罚它对错误类别过度自信。

还需要知道：均方误差也可以用于某些分类场景，但逻辑回归通常使用交叉熵作为训练目标。

九、把所有知识连成一条主线

现在，我们将前面学过的内容串起来。

假设我们要训练一个垃圾邮件分类器。

第一步：准备数据 收集邮件特征和对应标签。

第二步：建立模型 逻辑回归先根据特征进行线性计算，再使用 Sigmoid 得到正类概率。

第三步：计算损失 根据预测概率和真实标签计算交叉熵。

第四步：计算梯度 计算损失对模型参数的梯度，了解在当前参数附近如何调整参数。

第五步：更新参数 使用梯度下降调整参数。

第六步：重复训练 不断预测、计算损失和更新参数，让模型逐步学会数据中的规律。

第七步：评估模型 使用独立的验证集和测试集，检查模型对新数据的表现。

整条主线可以概括为：

数据 → 模型 → 预测 → 损失 → 梯度 → 参数更新 → 评估

一、Softmax：如何让模型进行多分类?

1. 为什么需要 Softmax?

回忆之前学过的逻辑回归。

对于二分类问题，我们可以使用 Sigmoid，把模型的输出转换成一个 0～1 之间的数，作为正类概率的估计。

但是，如果我们要判断一张图片究竟是猫、狗还是鸟，就需要处理多个类别。

这时可以使用 Softmax。

假设模型对一张图片输出了三个分数：[z=[3,2,1]]

这三个分数叫作 **Logits（原始分数）**。

注意：它们不是概率，因为可能为负数，也不要求加起来等于 1。

Softmax 的作用就是将这些分数转换为一组概率。

公式是：

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

不用急着记住公式的每一个细节，只需要理解两个步骤：

- 对每个分数取指数 (e^{z_i})，使结果为正。
- 将每个指数结果除以所有指数结果之和，使所有概率加起来等于 1。

例如，输入：[z=[3,2,1]]

经过 Softmax 后，得到的概率约为：[P=[0.665,0.245,0.090]]

因此，模型认为第一个类别最有可能。

这里有一个容易误解的地方

Softmax 会放大较高分数相对于较低分数的优势，但它不是简单地把分数差距乘大。

更准确地说，它通过指数变换和归一化，把原始分数转换成概率分布。

一句话总结：Logits 是模型输出的原始分数，Softmax 将它们转换成总和为 1 的类别概率。

2. 多分类交叉熵是什么?

Softmax 得到概率后，我们还需要判断预测得好不好。

假设真实类别是狗。

模型 A 预测：

类别	概率
猫	0.1
狗	0.8
鸟	0.1

模型 B 预测：

类别	概率
猫	0.7
狗	0.2
鸟	0.1

显然，模型 A 给真实类别“狗”分配了更高的概率，因此它的预测更好。

多分类交叉熵可以衡量这种预测与真实标签之间的差异。

公式是：

$$L = -\sum_{i=1}^{C} y_i \log p_i$$

这里：

- (y_i)：真实标签对应的值。
- (p_i)：模型给第 (i) 个类别分配的概率。

对于普通的单标签多分类任务，真实标签通常采用 One-hot 形式。

例如，类别顺序为猫、狗、鸟：[y=[0,1,0]]

由于只有狗对应的标签为 1，损失函数可以简化为：[L=-\log p_{\text{狗}}]

所以：

- 如果模型给狗的概率是 0.8，损失较小。
- 如果模型给狗的概率是 0.2，损失较大。
- 如果模型给狗的概率非常接近 0，损失会非常大。

这就鼓励模型给真实类别分配更高的概率。

把 Softmax 和交叉熵联系起来

它们的分工不同：

概念	作用
Logits	模型输出的原始分数
Softmax	将原始分数转换为类别概率
交叉熵	衡量模型对真实类别的预测质量
梯度下降	根据损失对参数的梯度更新模型参数

训练流程可以理解为：

输入数据 → 模型产生 Logits → Softmax 转换概率 → 交叉熵计算损失 → 计算梯度 → 更新参数。

补充一个实际开发中很重要的细节：在 PyTorch 中，`CrossEntropyLoss` 通常直接接收 Logits，而不是提前计算好的 Softmax 概率，因为它内部会采用数值稳定的方式完成相关计算。

你需要掌握的程度

目前你只需要理解：

1. Softmax 解决什么问题。
2. 交叉熵为什么能够评价分类预测。
3. 两者如何配合梯度下降完成训练。

暂时不需要深入研究对数函数的性质或交叉熵的完整数学推导。

二、泛化与过拟合：为什么模型可能只会做训练题?

1. 什么是泛化能力?

假设我们收集了 1000 张猫和狗的图片，用它们训练一个分类模型。

训练完成后，我们再拿出 200 张训练时没有使用过的图片，让模型判断它们是猫还是狗。

如果模型在这 200 张新图片上仍然表现良好，说明它具备较好的泛化能力。

泛化能力，就是模型将从训练数据中学到的规律应用到未见过的数据上的能力。

这里要区分三个数据集：

数据集	作用
训练集	用于学习模型参数
验证集	用于选择模型、调整超参数和决定何时停止训练
测试集	用于最终评估选定的模型

为什么需要三个数据集?

因为我们不能只根据训练集表现判断模型好坏。

如果反复根据测试集的结果修改模型，最终也可能间接地对测试集产生过度适应，使评估结果过于乐观。

因此，测试集应尽量留到模型选择完成后再使用。

2. 什么是过拟合?

假设一个模型在训练集上的准确率是 99%，在测试集上的准确率却只有 65%。

这说明模型可能过拟合了。
即:过度适配训练数据

你可以把它理解成：

学生把练习册里的答案全部背了下来，平时练习几乎全对，但是考试换了一批题目，成绩就大幅下降。

模型也可能把训练数据中的偶然特征甚至噪声当成规律。

例如，模型训练时发现猫的图片经常有绿色草地背景，就错误地学到“绿色背景意味着猫”。

但真正遇到一只在室内的猫时，它可能判断错误。

模型学到的不是可靠的猫的特征，而是训练数据中一个偶然出现的关联。

这就是过拟合的直观含义。

3. 欠拟合是什么?

反过来，如果模型太简单，连训练数据中的基本规律都没有学到，就可能出现欠拟合。

例如，真实的数据规律明显是弯曲的，你却使用一条直线去拟合。

无论如何调整直线的斜率和截距，都难以准确描述数据。

这种情况下，训练集和测试集的表现通常都不好。

我们可以先记住：

情况	训练集表现	测试集表现
欠拟合	通常较差	通常较差
拟合较合理	较好	较好
过拟合	较好	相对较差

这里的“较好”和“较差”需要结合具体任务与指标判断。

重点：我们的目标不是让训练集上的结果无限接近完美，而是让模型能够在新数据上保持良好表现。

三、正则化与早停：怎样减少过拟合?

我们已经知道过拟合的问题，那么应该怎么处理?

先记住四类常见方法：

- 增加高质量训练数据。
- 限制模型不必要的复杂性。
- 使用正则化。
- 使用早停。

1. 正则化是什么?

回忆之前学过的损失函数。

原本，我们希望预测误差尽可能小

但是，如果只关注训练误差，模型可能为了拟合每个训练样本而变得过于复杂。

于是，我们可以增加一个惩罚项：

$$L_{\text{total}} = L_{\text{MSE}} + \lambda \sum_j w_j^2$$

这是一种常见的 L2 正则化形式。

其中：

- L_{MSE}：预测误差。
- $\sum_j w_j^2$：对模型权重施加的惩罚。
- λ：控制惩罚强度的超参数。

现在，模型不再只考虑预测误差，还需要考虑权重大小带来的惩罚。

=

直观上，我们是在告诉模型：

“你可以学习数据中的规律，但不要仅仅为了拟合训练数据而依赖不必要的大权重。”

不过，**L2 正则化并不是直接禁止模型复杂，也不能保证一定消除过拟合。**它是一种有助于改善泛化能力的训练方法。

2. 正则化强度应该多大?

如果 (\lambda) 太小，正则化作用可能不够明显。

如果 (\lambda) 太大，模型又可能受到过强的限制，无法充分学习真实规律，从而欠拟合。

因此，我们需要选择合适的 (\lambda)。

它属于超参数，而不是模型训练时直接学习的权重。

通常，我们会尝试不同的 (\lambda)，比较验证集表现，再选择更合适的值。

3. 什么是早停?

假设模型训练过程中出现以下变化：

训练轮数	训练集损失	验证集损失
1	0.80	0.90
5	0.40	0.50
10	0.20	0.30
20	0.08	0.35
30	0.02	0.60

前期，训练集损失和验证集损失都在下降，说明模型正在学习有效规律。

但后来，训练集损失继续下降，验证集损失却开始上升。

这说明模型可能开始过度适应训练数据。

此时，我们可以采用早停：

1. 训练时持续观察验证集损失。
2. 保存验证集表现最好的模型参数。
3. 当验证集表现持续恶化时，停止训练。
4. 恢复到此前保存的最佳参数。

注意，实际训练时不一定在验证集损失第一次上升时就停止，因为短期波动不一定意味着过拟合。

正则化与早停有什么区别?

正则化主要改变训练目标，让模型在学习数据规律时受到额外约束。

早停则主要控制训练过程，在验证集表现较好的阶段停止继续训练。

两者可以配合使用。

四、偏差与方差：理解欠拟合和过拟合背后的原因

这是我们目前正在学习的内容。

先记住两个直观概念。

1. 偏差（Bias）

偏差可以粗略理解为：模型的预测与真实规律之间存在的系统性偏离。

假设真实关系是一条弯曲的曲线，而你使用直线模型去拟合。

即使更换不同的训练数据，模型仍然只能学到比较简单的直线关系。

它可能始终无法准确描述真实规律。

这就体现了高偏差的特点。

高偏差通常与欠拟合有关。

2. 方差（Variance）

方差关注的是：如果我们更换训练数据，重新训练模型，模型学到的结果会变化多少。

假设一个模型非常复杂。

第一次训练得到一条曲线，第二次训练更换了一批样本后，曲线的形状就发生了明显变化。

这说明模型对训练数据比较敏感。

这就是高方差的直观特点。

高方差通常与过拟合有关。

3. 偏差和方差怎样帮助我们调整模型?

观察到的问题	可能的原因	可以尝试的方向
训练集表现差，测试集也差	高偏差、欠拟合	增加模型表达能力、改善特征、检查训练过程
训练集表现好，测试集明显差	高方差、过拟合	正则化、早停、增加训练数据、数据增强

这里需要纠正一种常见的误解：

模型越复杂，训练集上的表现通常不会变差，但测试集上的表现不一定一直变好。

当模型从过于简单逐渐变得复杂时，测试表现可能改善。

但如果模型继续变复杂，开始拟合训练数据中的噪声，测试表现就可能变差。

这也是为什么不能仅仅追求训练集上的高准确率。

KNN 与决策树

一、KNN（K-Nearest Neighbors，K 近邻算法）

1. 核心思想

KNN 的核心思想：

一个新样本与哪些历史样本最相似，就可以参考这些历史样本的结果来预测它的结果。

例如：

现在有一些学生的历史数据，包括：

- 每天学习时间
- 每天练习题数量
- 是否通过考试

来了一个新学生，我们不知道他是否能通过考试。

KNN 会寻找与他最相似的 K 个历史学生，根据这些学生的考试结果预测新学生是否通过。

2. KNN 的预测过程

主要分为四步：

1. 计算新样本与训练集中各个样本之间的距离。
2. 找到距离最近的 K 个样本。
3. 根据这 K 个样本的标签进行判断。
4. 得到最终预测结果。

3. KNN 的分类与回归

KNN 可以用于分类，也可以用于回归。

分类任务：多数投票

例如，K=5：

- A 类：3 个
- B 类：2 个

预测结果为 A 类。

回归任务：取平均值或加权平均值

例如，最近的 3 套房屋价格：

- 100 万
- 110 万
- 120 万

预测价格：

(100+110+120)/3=110

4. K 值的影响

K 表示参考多少个最近邻居。

- K 太小：容易受到个别异常样本、噪声的影响，预测不稳定。
- K 太大：可能把距离较远、相似性较低的样本也纳入判断，导致预测效果下降。

因此，K 并不是越大越好，也不是越小越好。

通常通过验证集选择合适的 K。

5. 距离计算

常见方法：欧氏距离。

二维情况下：

欧氏距离：

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

距离越小，通常表示两个样本在所选特征上越相似。

6. 特征标准化

为什么需要标准化?

假设使用两个特征：

- 房屋面积：几十到几百平方米。
- 卧室数量：1～5 间。

两个特征的数值范围差异较大，直接计算距离时，面积可能对距离产生过大的影响。

例如，常见的 Z-score 标准化是：

$$x' = \frac{x - \mu}{\sigma}$$

其中：

- x：原始特征值。
- μ：该特征在训练数据中的均值。
- σ：该特征在训练数据中的标准差。
- x'：标准化后的特征值。

注意：均值和标准差只使用训练集计算，再应用到验证集和测试集，避免数据泄漏。

7. KNN 的训练特点

KNN 被称为一种惰性学习算法。

原因：

- 训练阶段：主要保存训练样本及其标签。
- 预测阶段：计算距离、寻找最近邻居、进行投票或取平均。

它不像线性回归那样，通常通过梯度下降学习一组模型参数。

8. KNN 的优缺点

优点：

- 思路简单，容易理解。
- 可以处理分类和回归。
- 不需要传统意义上复杂的参数训练过程。

缺点：

- 训练数据量大时，预测可能较慢。
- 对特征尺度比较敏感，通常需要标准化。
- 对 K 值、距离度量和异常样本比较敏感。
- 高维数据中，距离可能难以有效体现样本相似性。

9. KNN 的核心问题

怎样定义有意义的相似性？

重点关注：

- 特征是否有用。
- 是否需要标准化。
- 使用什么距离度量。
- K 值如何选择。

二、决策树（Decision Tree）

1. 核心思想

决策树的核心思想：

从训练数据中学习一系列判断规则，根据这些规则逐步判断，最终得到预测结果。

例如，预测学生是否通过考试：

```
每天练习题数量 > 20?
├── 否 → 未通过
└── 是 → 每天学习时间 > 2小时?
          ├── 否 → 未通过
          └── 是 → 通过
```

预测时，从根节点开始，根据条件选择对应分支，直到到达叶节点。

2. 决策树的基本组成

- 根节点：最上方的第一个判断条件。
- 内部节点：中间继续进行判断的节点。
- 分支：判断后选择的路径。
- 叶节点：最终预测结果，不再继续分裂。

可以把决策树理解为一系列有先后顺序的条件判断。

3. 决策树如何训练?

决策树不是人为提前写好所有判断规则，而是根据训练数据学习出来的。

基本过程：

1. 将所有训练样本放入根节点。
2. 尝试不同特征和划分条件。
3. 评估不同划分的效果。
4. 选择较好的划分条件。
5. 将样本分配到不同子节点。
6. 对子节点重复上述过程。
7. 满足停止条件后结束分裂，形成叶节点。

核心目标：

让划分后的子节点尽可能纯净，即同一个节点中的样本尽量属于同一类别。

4. 什么是纯度?

例如有 100 个学生：

节点	通过人数	未通过人数	纯度
A	100	0	非常纯
B	50	50	很混杂
C	90	10	比 B 更纯

分类任务中：

- 类别越集中，节点越纯。
- 类别越混杂，节点越不纯。

5. 信息熵（Entropy）

信息熵用于衡量节点中的不确定性或类别混杂程度。

对于分类问题，信息熵的公式是：

$$H = -\sum_{i=1}^{C} p_i \log_2 p_i$$

其中：

- C：类别数量。
- p_i：第 i 类样本所占的比例。

对于二分类问题：

通过人数	未通过人数	信息熵
100	0	0
90	10	约 0.469
50	50	1

结论：

- 熵越低，节点越纯。
- 熵越高，节点越混杂。
- 所有样本都属于同一类别时，熵为 0。
- 二分类中，两类样本各占一半时，熵达到最大值 1。

6. 信息增益（Information Gain）

信息增益用于衡量一次划分让不确定性减少了多少。

$$\text{信息增益} = H(\text{划分前}) - H(\text{划分后})$$

其中，划分后的熵需要按照各子节点样本数量进行加权。

例如：

划分前：

- 50 人通过。
- 50 人未通过。
- 信息熵为 1。

方案 A：

- 左节点：50 人全部通过。
- 右节点：50 人全部未通过。

划分后熵为 0。

[IG=1-0=1]

方案 B：

- 左节点：25 人通过、25 人未通过。
- 右节点：25 人通过、25 人未通过。

划分后熵仍然为 1。

[IG=1-1=0]

因此方案 A 更好。

结论：

信息增益越大，说明这次划分减少的不确定性越多。

补充：

- ID3：使用信息增益。
- C4.5：使用信息增益率。
- CART：分类树常使用基尼不纯度。

7. 决策树的过拟合

如果决策树不断分裂，可能变得非常复杂，甚至记住训练数据中的噪声和偶然规律。

结果：

- 训练集表现很好。
- 测试集表现较差。

这就是过拟合。

8. 如何避免决策树过拟合?

常见方法：

1. 限制最大深度：避免树变得过深。
2. 限制叶节点最少样本数：避免根据少量样本建立分支。
3. 剪枝：删除不必要的分支。
4. 使用验证集选择超参数。
5. 增加高质量训练数据。

9. 决策树的分类与回归

分类树：

叶节点输出类别。

例如：

- 通过
- 未通过

回归树：

叶节点输出数值。

例如，某个叶节点中的房屋价格为：

100 万、110 万、120 万。

预测值可以取平均数：

$$
\hat y=\frac{100+110+120}{3}=110
$$

10. 决策树的优缺点

优点：

- 判断过程直观，容易解释。
- 通常不需要特征标准化。
- 可以处理分类和回归任务。
- 可以学习特征之间的非线性关系和组合判断规则。

缺点：

- 容易过拟合。
- 对训练数据变化可能比较敏感。
- 单棵决策树的预测能力有时有限。

11. 决策树的核心问题

怎样找到合适的判断规则，并控制树的复杂度?

重点关注：

- 特征选择。
- 划分条件。
- 信息熵、信息增益、基尼不纯度。
- 最大深度。
- 叶节点限制。
- 剪枝。

三、KNN 与决策树的区别

对比维度	KNN	决策树
核心思想	寻找相似样本	学习判断规则
预测方式	找最近的 K 个邻居	沿着树的分支进行判断
训练阶段	主要保存训练数据	学习特征划分规则
预测阶段	计算距离	执行条件判断
是否需要标准化	通常需要考虑	通常不需要
主要超参数	K、距离度量	最大深度、叶节点限制等
主要风险	距离不合理、噪声影响	过拟合、树结构不稳定
分类	可以	可以
回归	可以	可以

最重要的区别

KNN：看与你相似的人，参考他们的结果。

决策树：学习一系列判断规则，根据规则得出结果。

四、本节需要掌握的知识

学习完本节，应该能够回答：

1. KNN 的 K 表示什么?
2. KNN 如何完成分类和回归?

3. 为什么 KNN 通常需要特征标准化?
4. 为什么 KNN 被称为惰性学习算法?
5. 决策树的根节点、内部节点、分支和叶节点分别是什么?
6. 决策树如何选择划分条件?
7. 什么是信息熵?熵越高意味着什么?
8. 什么是信息增益?
9. 为什么决策树容易过拟合?
10. 怎样限制决策树的复杂度?

一句话总结

KNN 通过样本之间的相似性进行预测；决策树通过训练数据学习判断规则进行预测。两者都可以解决分类和回归问题，但预测机制完全不同。

第12课：随机森林（Random Forest）完整知识笔记

一、随机森林是什么?

随机森林的核心思想是：

训练多棵具有一定差异的决策树，然后综合它们的预测结果，得到最终结果。

举个例子：

我们要预测一个学生是否能通过考试。

- 决策树1：预测通过。
- 决策树2：预测通过。
- 决策树3：预测未通过。
- 决策树4：预测通过。
- 决策树5：预测未通过。

最终：

- 通过：3票。
- 未通过：2票。

因此，随机森林预测该学生通过考试。

两种任务的结果汇总方式:

任务	汇总方式
分类	多数投票

任务	汇总方式
回归	多棵树预测值取平均

为什么不只训练一棵树?

因为单棵决策树容易受到训练数据变化的影响，可能过拟合。多棵具有差异的树进行综合判断，通常能降低预测的波动，提高模型稳定性。

但要注意：**不是树之间差异越大越好，而是每棵树都要有一定预测能力，同时树与树之间要有足够的差异。**

二、随机森林的两个随机机制

这是随机森林最重要的知识点。

```
随机：

1）数据随机

2）每个节点的候选特征随机
```

1. Bootstrap：对训练样本进行有放回抽样

假设原始训练集有100条数据。

训练第一棵树时，从这100条数据中有放回地抽取100次。

例如：

原始数据：1 2 3 4 5 6 7 8 9 10 ...

抽样结果：1 2 2 3 5 5 6 8 8 10 ...

你会发现：

- 有些样本被抽取多次。
- 有些样本一次也没有被抽到。
- 抽取100次，不代表得到100条不同的数据。

数学上：

某条样本一次都没有被抽中的概率：

$$P = \left(1 - \frac{1}{100}\right)^{100} \approx 36.8\%$$

至少被抽中一次的概率：

$$1 - 36.8\% = 63.2\%$$

所以，100条原始数据经过100次有放回抽样，期望得到约63.2条不同样本。

每棵树都独立进行 Bootstrap，因此不同树使用的训练数据会有所不同。

2. 特征随机选择：每个节点随机选择候选特征

假设我们有5个特征：

- A：做题数量。
- B：学习时间。
- C：睡眠时间。
- D：上课情况。
- E：作业完成情况。

在某个节点需要分裂时，随机选择部分特征作为候选。

例如：

```
所有特征：A B C D E

随机候选：A C E
```

然后比较 A、C、E 的划分效果，选择其中最好的一个。

注意：

不是随机抽到 A，就直接用 A 分裂；而是先随机确定候选集合，再从候选集合中选择划分效果最好的特征。

这样做是为了避免所有树都过度依赖相同的强势特征。

三、重点：一棵决策树是如何逐步形成的？

这是我们讨论时间最长、也是最容易混淆的地方。

1. 一棵树不只选择一个特征

一棵决策树有多个节点。

每个需要分裂的节点，都可以重新选择候选特征，然后选出一个最合适的特征进行分裂。

例如：

```
                A：做题数量 > 20？
                  /       \
                 是         否
                /             \
B：学习时间 > 2小时？  C：睡眠时间 > 6小时？
   /       \              /       \
  ...      ...          ...       ...
```

过程是:

1> 不放回地抽取了100条数据,除去重复抽取的,假设得到62条有效数据

2> 我们在共有的A-E中抽取3个候选特征,得:A,C,D

3> 3个特征对比,发现A特征划分效果做好---取A

4> 在A的符合"是"的数据中(假设62条数据中有42条符合),我们再进行2>操作,假设得到B,C,D.取效果最好的B."否"也是这样操作

5> 如此循环...

因此：

- 一个节点通常选择一个特征进行分裂。
- 一棵树可以有很多节点。
- 一棵树可以使用多个不同特征。
- 同一个特征可以在不同节点重复使用。

2. 同一个特征为什么可以重复使用？

假设根节点选择了 A：

```
A：做题数量 > 20？
```

随后左节点又选择了 A：

```
A：做题数量 > 40？
```

这是完全允许的。

因为两个节点处理的样本范围不同，判断条件也可以不同。

例如：

- 根节点：使用全部100条样本，判断做题数量是否大于20。
- 左节点：只使用根节点分配给左侧的样本，进一步判断做题数量是否大于40。

同一个特征可以重复使用，但每次使用时的划分阈值可能不同。

3. 最重要：样本是怎样在树中流动的？

这是你之前提出疑惑最多的地方。

假设 Bootstrap 后，这棵树有100条训练样本。

根节点使用全部100条样本。

现在根节点选择：

```
A：做题数量 > 20？
```

分裂结果：

```
                根节点（100条）
              A：做题数量 > 20？
                 /        \
                /          \
        左节点（60条）    右节点（40条）
```

接下来：

- 左节点只能使用自己的60条样本。
- 右节点只能使用自己的40条样本。

假设右节点继续分裂，它只能使用自己的40条样本，不能把左节点的60条拿过来。

为什么？

因为这60条样本已经被根节点分配到了另一条分支，不属于右节点的样本范围。

现在即使右节点再次选择 A，也只是在自己的40条样本中进行判断。

这里一定要区分两个操作：

操作	作用
Bootstrap	在每棵树开始训练时，对原始训练集有放回抽样
节点分裂	根据判断条件，把当前节点的样本分配到不同子节点

节点分裂不是重新抽样！

这是我们本次讨论中最重要的结论。

四、决策树什么时候停止分裂？

决策树不会无限分裂。

常见停止条件包括：

1. **节点已经足够纯净**：例如节点中100个样本全部属于同一个类别。
2. **达到最大深度**：例如设置 `max_depth=3`，树不能继续向下生长。
3. **样本数量太少**：例如设置 `min_samples_split=10`，少于10个样本的节点不允许继续分裂。
4. **划分收益不足**：继续分裂不能带来足够的效果提升。

停止分裂后，节点成为叶节点。

分类任务中，叶节点通常输出该节点样本中的多数类别；回归任务中，通常输出该节点目标值的平均值。

五、随机森林完整训练流程

把前面的内容串起来：

```
准备原始训练集
        |
        v
训练第一棵决策树：

Bootstrap有放回抽样
        |
        v
从根节点开始训练
        |
        v
随机选择部分候选特征
        |
        v
从候选特征中选择最佳划分
        |
        v
根据判断条件划分样本
        |
```

```
          v
对子节点重复上述过程
          |
          v
达到停止条件，树训练完成
          |
          v
训练其他决策树
          |
          v
得到多棵决策树
          |
          v
分类投票 / 回归取平均
          |
          v
最终预测结果
```

六、OOB：袋外数据

由于 Bootstrap 是有放回抽样，因此每棵树都会有一些原始训练样本没有被抽到。

这些没有参与某棵树训练的样本，称为这棵树的袋外数据（Out-of-Bag，OOB）。

例如：

- 原始训练集有100条数据。
- 某棵树抽样后，只有约63条不同样本。
- 剩余约37条没有被抽到。

这37条就可以作为这棵树的袋外数据。

可以让这棵树预测这些样本，再综合多棵树的袋外预测，估计模型的泛化表现。

因此，随机森林可以利用 OOB 估计模型效果，而不一定需要额外划分验证集。

七、随机森林的优缺点

优点：

- 通常比单棵决策树更加稳定。
- 能用于分类和回归。
- 能处理多种特征。
- 能降低单棵树过拟合带来的风险。

- 可以估计特征重要性。

缺点：

- 多棵树需要更多内存。
- 训练和预测成本通常高于单棵树。
- 整体可解释性不如单棵决策树。
- 仍然需要合理调整超参数。

八、本节最终知识框架

把随机森林记成三个层次：

层次	核心操作	解决的问题
数据层	Bootstrap	让不同树看到不同的样本
特征层	节点随机选择候选特征	让不同树具有差异
预测层	投票或平均	综合多棵树的结果

最值得记住的四句话：

1. Bootstrap 是对样本有放回抽样，不是对特征抽样。
2. 每个节点随机选择候选特征，再从中选择最好的特征进行分裂。
3. 一棵树可以使用多个特征，同一个特征也可以重复使用。
4. 节点分裂是划分当前样本，不是重新抽样；子节点只能使用分配给自己的样本。
